

👤

lyusupov fix against regression on low RAM SoCs

✓

3 days ago

⋮

🕒

Name		Name	Last commit date
📁 ..			
📁 SkyView	SkyView: current state is good ...		2 months ago
📁 SkyWatch	fix against regression of SkyWat...		2 months ago
📁 SoftRF	illustrations for the Prime3 wiki ...		last week
📁 UAT_Receiver	Copyright extension onto year ...		9 months ago
📁 UATbridge	Copyright extension onto year ...		9 months ago
📁 iLogger	Copyright extension onto year ...		9 months ago
📁 libraries	fix against regression on low R...		3 days ago
📄 README.md	fix against regression on low R...		3 days ago

SoftRF firmware build instructions

- [NodeMCU](#)
- [ESP32](#)
- [Raspberry Pi](#)
- [CC13X0](#)
- [STM32](#)
- [ASR650x](#)
- [nRF52840](#)
- [SAMD21](#)
- [RP2040](#)

NodeMCU

1. Follow [these official instructions](#) to install Arduino IDE and latest Arduino ESP8266 Core
2. Become familiar with IDE and **NodeMCU** by building and uploading of a basic **Blink** sketch:

File -> Examples -> ESP8266 -> Blink

then

Sketch -> Upload

3. When you are done with the lesson, close your **Arduino** application
4. open <My Documents> (Windows), <Home> (Linux) or <Documents> (MacOS) directory
5. create **Arduino** sub-directory
6. transfer full content of **SoftRF** and **libraries** GitHub folders into the sub-directory:

[SoftRF](#) --> <My Documents> /Arduino/SoftRF
[libraries](#) --> <My Documents> /Arduino/libraries
7. start **Arduino** application again
8. open **SoftRF** sketch from *File -> Open* menu
9. Select *Tools -> Board -> NodeMCU 1.0*
10. in *Tools -> lwIP Variant -> v2 Higher Bandwidth*
11. *Sketch -> Upload*

ESP32

1. Follow [these official instructions](#) to install Arduino IDE and a **certain version** of Arduino ESP32 Core
For **ESP32**: [1.0.5](#)
For **ESP32-S3**: [2.0.9](#)
For **ESP32-C3**: [2.0.9](#)
2. Become familiar with IDE and **DoIt ESP32 DevKit** by building and uploading of a basic **Blink** sketch:

```
int ledPin = 2; // use pin 14 for TTGO T-Beam rev.05 or higher
```

```
void setup()  
{
```



```

    pinMode(ledPin, OUTPUT);
    Serial.begin(115200);
}

void loop()
{
    Serial.println("Hello, world!");
    digitalWrite(ledPin, HIGH);
    delay(500);
    digitalWrite(ledPin, LOW);
    delay(500);
}

```

3. When you are done with the lesson, close your **Arduino** application
4. open <My Documents> (Windows), <Home> (Linux) or <Documents> (MacOS) directory
5. create **Arduino** sub-directory
6. transfer full content of **SoftRF** and **libraries** GitHub folders into the sub-directory:

[SoftRF](#) --> <My Documents> /Arduino/SoftRF
[libraries](#) --> <My Documents> /Arduino/libraries

7. start **Arduino** application again
8. open **SoftRF** sketch from *File -> Open* menu
9. For **ESP32**:
 Select *Tools -> Board -> ESP32 Dev Module*
 For **ESP32-S3**:
 Select *Tools -> Board -> ESP32-S3 Dev Module*
 For **ESP32-C3**:
 Select *Tools -> Board -> ESP32-C3 Dev Module*
10. Select *Tools -> Flash Mode -> DIO*
11. Select *Tools -> Flash Size -> 4MB*
12. Select *Tools -> Partition Scheme -> Minimal SPIFFS*
13. Select *Tools -> Flash Frequency -> 80MHz*
14. Select *Tools -> CPU Frequency -> 80MHz*
15. Select *Tools -> PSRAM -> Enabled*
16. *Sketch -> Upload*

Raspberry Pi

Although CLI application's source code for Raspberry Edition is located [at the same place](#) and is shared with other SoftRF platforms - build instructions for the code are different.

This build has to be done on a Raspberry Pi host.

A known good Raspbian OS version that fits for this purpose is "Stretch".

Make sure that basic development packages (such as: binutils, g++, GNU make & etc) are installed.

1. transfer full content of **SoftRF** and **libraries** GitHub folders into a temporary build directory:

```
SoftRF --> <your path> /SoftRF
libraries --> <your path> /libraries
```

2. change directory on <your path> /SoftRF and execute **make** as follows:

```
pi@raspberrypi: $ make pi
(cd ../libraries/bcm2835/src/.. ; ./configure ; make)
checking for a BSD-compatible install... /usr/bin/install -c
checking whether build environment is sane... yes
checking for a thread-safe mkdir -p... /bin/mkdir -p
checking for gawk... no
< ... skipped ... >
```



As a result of the build, two program binaries will become created:

- **SoftRF** - the program code designed to work with Raspberry Pi's primary SPI bus (SPI 0);
- **SoftRF-aux** - the same program but to run over auxiliary SPI bus (SPI 1).

CC13X0

[Energia](#) IDE has to be pre-installed first.

1. open <Home> directory
2. create **Energia** sub-directory
3. transfer full content of **SoftRF** and **libraries** GitHub folders into the sub-directory:

```
SoftRF --> <My Documents> /Energia/SoftRF
libraries --> <My Documents> /Energia/libraries
```

4. start **Energia** application
5. open **SoftRF** sketch from *File -> Open* menu
6. [install support](#) for TI CC13XX family with *Tools -> Board -> Boards Manager...*

7. Select *Tools* -> *Board* -> *LaunchPad w/ CC1310 EMT (48MHz)*

8. Select *Tools* -> *Port* -> <your XDS110 port device name>

9. Select *Tools* -> *Programmer* -> *dslite*

10. try to build and upload using *Sketch* -> *Upload*

10a. or, to build the firmware using shell prompt, do:

```
$ make cc
energia --verify --verbose-build SoftRF.ino
Picked up JAVA_TOOL_OPTIONS:
Loading configuration...
Initializing packages...
Preparing boards...
Verifying...

< ... skipped ... >
```



11a. inspect **/tmp** directory and sub-folders, take **SoftRF.ino.elf** file and use it with TI's **Uniflash** or **Flash Programmer 2**.

12a. (Optional) if you want to use [cc2538-bsl.py](#) tool to put the firmware into flash memory of CC1310 IC via **boot loader**, run **make ihex**:

```
$ make ihex

< ... skipped ... >
```



STM32

You will need to have an ST-LINK/V2 USB adapter connected in order to put the firmware into your hardware's flash memory.

1. Follow [these official instructions](#) to install Arduino IDE and **certain version** of Arduino STM32

[SoftRF](#) / [software](#) / [firmware](#) / [source](#) /

[↑](#) Top

For **STM32WLE5**: [2.4.0](#)

For **STM32F103C8** "Blue Pill": [2.4.0](#)

2. open <My Documents> (Windows), <Home> (Linux) or <Documents> (MacOS) directory

3. create **Arduino** sub-directory

4. transfer full content of **SoftRF** and **libraries** GitHub folders into the sub-directory:

[SoftRF](#) --> <My Documents> /Arduino/SoftRF
[libraries](#) --> <My Documents> /Arduino/libraries

5. start **Arduino** application

6. open **SoftRF** sketch from *File -> Open* menu

7. For **S76G**:

Select *Tools -> Board -> Nucleo_64*

Select *Tools -> Board part number -> Nucleo L073RZ*

Select *Tools -> U(S)ART support -> Enabled (no generic 'Serial')*

Select *Tools -> USB support (if available) -> CDC (generic 'Serial' supersede U(S)ART)*

Select *Tools -> USB speed (if available) -> Low/Full Speed*

For **STM32WLE5**:

Select *Tools -> Board -> Generic STM32WL series*

Select *Tools -> Board part number -> Generic WLE5CCUx*

Select *Tools -> U(S)ART support -> Enabled (generic 'Serial')*

For **STM32F103C8** "Blue Pill":

Select *Tools -> Board -> Generic STM32F1 series*

Select *Tools -> Board part number -> BluePill F103CB (or C8 with 128k)*

Select *Tools -> U(S)ART support -> Enabled (generic 'Serial')*

Select *Tools -> USB support (if available) -> CDC (no generic 'Serial')*

Select *Tools -> USB speed (if available) -> Low/Full Speed*

8. Select *Tools -> Optimize -> Smallest (-Os default)*

9. Select *Tools -> C Runtime library -> Newlib Nano (default)*

10. For **S76G "Dongle"**:

Select *Tools -> Upload method -> STM32CubeProgrammer (DFU)*

Select *Tools -> Port -> <your Dongle's DFU device name>*

For **STM32WLE5**:

Select *Tools -> Upload method -> STM32CubeProgrammer (SWD)*

Select *Tools -> Port -> <your ST-LINK/V2 port device name>*

For **S76G "SkyWatch"** or **STM32F103C8** "Blue Pill":

Select *Tools -> Upload method -> STM32CubeProgrammer (SWD)*

Select *Tools -> Port -> <your ST-LINK/V2 port device name>*

11. try to build and upload using *Sketch -> Upload*

ASR650x

1. Follow [these official instructions](#) to install Arduino IDE and [latest stable Heltec CubeCell \(ASR650X\) Arduino Support](#) (1.4.0)

2. open <My Documents> (Windows), <Home> (Linux) or <Documents> (MacOS) directory

3. create **Arduino** sub-directory

4. transfer full content of **SoftRF** and **libraries** GitHub folders into the sub-directory:

SoftRF --> <My Documents> /Arduino/SoftRF

libraries --> <My Documents> /Arduino/libraries

5. start **Arduino** application

6. open **SoftRF** sketch from *File -> Open* menu

7. Select *Tools -> Board -> CubeCell-GPS (HTC-AB02S)*

8. Select *Tools -> LORAWAN_AT_SUPPORT -> OFF*

9. Select *Tools -> LORAWAN_RGB -> DEACTIVE*

10. Select *Tools -> LORAWAN_CLASS -> CLASS_A*

11. Select *Tools -> LORAWAN_ADR -> OFF*

12. Select *Tools -> LORAWAN_NETMODE -> ABP*

13. Select *Tools -> LORAWAN_Net_Reservation -> OFF*

14. Select *Tools -> LORAWAN_UPLINKMODE -> UNCONFIRMED*

15. Select *Tools -> LoRaWan Debug Level -> None*

16. Select *Tools -> LORAWAN_REGION -> REGION_EU433*

17. Select *Tools -> Port -> <your CubeCell port device name>*

18. try to build and upload using *Sketch -> Upload*

nRF52840

1. Follow [these official instructions](#) to install Arduino IDE and [latest stable Arduino Core for Adafruit Bluefruit nRF52 Boards](#) (1.3.0)

2. open <My Documents> (Windows) , <Home> (Linux) or <Documents> (MacOS) directory

3. create **Arduino** sub-directory

4. transfer full content of **SoftRF** and **libraries** GitHub folders into the sub-directory:

SoftRF --> <My Documents> /Arduino/SoftRF

libraries --> <My Documents> /Arduino/libraries

5. start **Arduino** application

6. open **SoftRF** sketch from *File -> Open* menu

7. Select *Tools* -> *Board* -> *Nordic nRF52840DK (PCA10056)*
8. Select *Tools* -> *Bootloader* -> *0.3.2 SoftDevice s140 6.1.1*
9. Select *Tools* -> *Debug* -> *Level 0 (Release)*
10. Select *Tools* -> *Port* -> <your nRF52840 port device name>
11. try to build and upload using *Sketch* -> *Upload*

SAMD21

1. Follow [these official instructions](#) to install Arduino IDE and latest Atmel SAMD Core
2. Become familiar with IDE and **Arduino M0** or **Zero** by building and uploading of a basic **Blink** sketch:

File -> *Examples* -> *01.Basics* -> *Blink*

then

Sketch -> *Upload*

3. When you are done with the lesson, close your **Arduino** application
4. open <My Documents> (Windows) , <Home> (Linux) or <Documents> (MacOS) directory
5. create **Arduino** sub-directory
6. transfer full content of **SoftRF** and **libraries** GitHub folders into the sub-directory:

[SoftRF](#) --> <My Documents> /Arduino/SoftRF

[libraries](#) --> <My Documents> /Arduino/libraries

7. start **Arduino** application again
8. open **SoftRF** sketch from *File* -> *Open* menu
9. *Sketch* -> *Upload*

RP2040

1. Follow [these official instructions](#) to install Arduino IDE and [latest stable Raspberry Pi Pico Arduino core, for all RP2040 boards](#) (3.2.1)
2. open <My Documents> (Windows) , <Home> (Linux) or <Documents> (MacOS) directory
3. create **Arduino** sub-directory

4. transfer full content of **SoftRF** and **libraries** GitHub folders into the sub-directory:

SoftRF --> <My Documents> /Arduino/SoftRF

libraries --> <My Documents> /Arduino/libraries

5. start **Arduino** application

6. open **SoftRF** sketch from *File -> Open* menu

7. For RAK11310 WisBlock Core:

Select *Tools -> Board -> Generic RP2040*

For Raspberry Pi Pico with Waveshare modules:

Select *Tools -> Board -> Raspberry Pi Pico*

8. Select *Tools -> Flash Size -> 2MB (Sketch: 1MB, FS: 1MB)*

9. Select *Tools -> USB Stack -> Adafruit TinyUSB*

10. try to build and upload using *Sketch -> Upload*